

Agentic AI Frameworks

Architecture, Orchestration Patterns, and Production Considerations

LangChain

LangGraph

CrewAI

AutoGen

Amazon Bedrock Agents

ReAct • Plan-Execute • Reflection • Supervisor

1. Introduction to Agentic AI

Traditional LLM usage follows a **prompt → response** pattern: single query, single completion. **Agentic AI** replaces this with an autonomous loop: the agent plans actions, invokes external tools, observes results, revises reasoning, and iterates until a goal is achieved without per-step human guidance. Key properties: **Autonomy** (self-directed execution), **Goal-Directed Behavior** (maintains objective across steps), **Tool Use** (invokes APIs, databases, code interpreters), **Memory** (short-term conversation buffer + long-term external store), **Reflection** (self-critique and error correction), and **Planning** (decomposes goals into ordered subtasks).

Dimension	Simple Prompt-Response	Agentic System
Execution	Single forward pass	Iterative observe → think → act loop
Tool access	None / static retrieval	Dynamic tool selection at runtime
State	Stateless	Scratchpad + memory + intermediate results
Latency	0.5–3s per call	5–120s per task (multi-call + I/O)
Cost	Fixed per query	Variable; typically 3–30x a single call

Cross-Reference — Docs 1 & 3

In RAG systems (Doc 1), retrieval is a fixed pipeline step. Agents treat vector database retrieval (Doc 3) as a *dynamic tool* — they can parameterize `top_k`, apply metadata filters, rewrite queries, and retry on failure. The vector DB becomes one tool among many in the agent's reasoning loop.

2. LangChain

LangChain

Type: Open-source orchestration License: MIT Stars: ~95k Version: v0.3+

LangChain is the most widely adopted LLM framework (~700 integrations). Architecture is built on four primitives: **Chains** (sequential prompt → LLM → parser compositions), **Agents** (LLM-driven tool-selection loops via `AgentExecutor`), **Tools** (`@tool`-decorated functions exposing a name, description, and JSON schema to the LLM), and **Memory** (pluggable backends: buffer, summary, vector store, entity). LCEL (v0.2+) replaces legacy chains with pipe-composition syntax supporting native async, streaming, and batching:

```
# LCEL RAG chain - pipe syntax: retriever | prompt | llm | parser
rag_chain = (
    {"context": vectorstore.as_retriever(k=5), "question": RunnablePassthrough()}
    | ChatPromptTemplate.from_template(ANSWER_TEMPLATE)
    | llm | StrOutputParser()
)
# AgentExecutor with tools
@tool
def search_knowledge_base(query: str) -> str: # wraps vector DB (Doc 3)
    return "\n".join([d.page_content for d in vectorstore.similarity_search(query, k=4)])
executor = AgentExecutor(agent=create_openai_tools_agent(llm, tools, prompt),
    tools=tools, max_iterations=10, handle_parsing_errors=True)
```

✓ Strengths

- 700+ integrations (LLMs, vector stores, APIs)
- Rapid prototyping; LCEL streaming/async built-in
- LangSmith native observability & eval
- Largest community and documentation

✗ Limitations

- Deep abstraction stacks — debugging non-trivial
- API churn: 3 major breaking migrations (v0.1→v0.3)
- `AgentExecutor` lacks graph-like flows; use `LangGraph` instead
- Python overhead measurable at high throughput

Use when: Rapid prototyping; standard RAG pipelines; broad integration needs. For stateful/production agentic flows, prefer `LangGraph`.

3. LangGraph

LangGraph

Type: Graph-based stateful orchestration (built on LangChain) **License:** MIT **Stars:** ~12k **Version:** v0.2+

LangGraph replaces LangChain's opaque `AgentExecutor` loop with an explicit **directed graph** where every routing decision is code. Core concepts: **Nodes** (Python functions receiving/returning typed state), **Edges** (fixed or conditional transitions), **State** (a `TypedDict` passed through the graph with reducer-merged updates), and **Checkpointing** (persistent state snapshots in SQLite/PostgreSQL/Redis enabling resumability and human-in-the-loop breakpoints).

```
class AgentState(TypedDict):
    messages: List[BaseMessage] # grows with each step
    final_answer: str # populated at END

def agent_node(state): return {"messages": [llm.invoke(state["messages"])]}
def tool_node(state): return {"messages": execute_tools(state["messages"][-1].tool_calls)}
def route(state) -> str: # conditional edge: call tools or end?
    return "tools" if state["messages"][-1].tool_calls else END

app = StateGraph(AgentState)
app.add_node("agent", agent_node); app.add_node("tools", tool_node)
app.set_entry_point("agent")
app.add_conditional_edges("agent", route); app.add_edge("tools", "agent")
app = app.compile(checkpointer=SqliteSaver.from_conn_string("db.sqlite"),
                  interrupt_before=["human_review"])
```

✓ Strengths

- Explicit state graph — every routing decision is auditable code
- Native cycles, parallel branches, sub-graphs
- Checkpointing enables HITL, resumability, time-travel debugging
- LangGraph Platform provides managed deployment + Studio UI

✗ Limitations

- Steeper learning curve; requires graph-design thinking upfront
- Newer ecosystem — fewer community examples vs. LangChain
- More boilerplate than `AgentExecutor` for simple tasks
- Inherits LangChain API churn

Use when: Production agentic workflows; HITL gates; complex multi-step reasoning with cycles; long-running resumable tasks. Preferred over `AgentExecutor` for anything non-trivial.

4. CrewAI

CrewAI

Type: Multi-agent role-based collaboration **License:** MIT **Stars:** ~28k **Version:** v0.80+

CrewAI models workflows as a **crew of role-specialized agents** collaborating toward a shared goal. Each **Agent** has a role, goal, backstory (LLM persona conditioning), tools, and an LLM assignment. **Tasks** define units of work with expected outputs. A **Crew** orchestrates execution using either `Process.sequential` (fixed order; output of task N feeds N+1) or `Process.hierarchical` (a manager LLM dynamically assigns tasks).

```
researcher = Agent(role="Senior AI Research Analyst",
                  goal="Find accurate technical information", backstory="...",
                  tools=[search_knowledge_base, web_search], llm=gpt4_llm)
writer = Agent(role="Technical Documentation Writer",
              goal="Transform findings into structured docs", backstory="...", llm=gpt4_llm)
crew = Crew(
    agents=[researcher, writer],
    tasks=[Task(description="Research LangGraph vs AutoGen", agent=researcher),
          Task(description="Write technical summary", agent=writer, context=[research_task])],
    process=Process.sequential
)
result = crew.kickoff(inputs={"topic": "agentic framework comparison"})
```

✓ Strengths

- Intuitive role-based design; maps to human team structures
- Minimal boilerplate: 3–5 agents in 30–50 lines
- Built-in short-term, long-term, entity, and contextual memory
- Role backstory effectively conditions agent specialization

✗ Limitations

- Less mature — fewer production deployments
- Complex conditional state management requires workarounds
- Sequential process cannot parallelize independent tasks natively
- Debugging agent-to-agent delegation is challenging

Use when: Tasks map to distinct expert roles; content generation pipelines; rapid multi-agent prototyping. Avoid for latency-critical flows or fine-grained state control requirements.

5. AutoGen (Microsoft)

AutoGen — Microsoft Research

Type: Multi-agent conversation framework **License:** MIT **Stars:** ~38k **Version:** v0.4+ (AutoGen Core + AgentChat)

AutoGen models agents as participants in **conversational threads**. The base `ConversableAgent` is subclassed into: **AssistantAgent** (LLM-backed; generates responses and code), **UserProxyAgent** (executes code in a Docker sandbox or solicits human input; `human_input_mode`: ALWAYS / TERMINATE / NEVER), and **GroupChatManager** (routes conversation turns among N agents using round-robin or LLM-driven speaker selection). First-class support for Docker-sandboxed code execution distinguishes AutoGen from other frameworks.

```
coder = autogen.AssistantAgent("Coder", llm_config=cfg,
    system_message="Expert Python developer. Write clean, tested code.")
reviewer = autogen.AssistantAgent("Reviewer", llm_config=cfg,
    system_message="Thorough code reviewer. Flag bugs and style issues.")
executor = autogen.UserProxyAgent("Executor", human_input_mode="NEVER",
    code_execution_config={"executor": DockerCommandLineCodeExecutor(
        image="python:3.11-slim", timeout=60, work_dir="./workspace")})
groupchat = autogen.GroupChat([coder, reviewer, executor], max_round=12,
    speaker_selection_method="auto")
manager = autogen.GroupChatManager(groupchat, llm_config=cfg)
executor.initiate_chat(manager, message="Build a RAG pipeline using pgvector + FastAPI")
```

✓ Strengths

- Research-grade flexibility; highly configurable agent types
- First-class Docker-sandboxed code generation + execution
- Azure OpenAI / Azure AI Search native integration
- Multi-agent debate and consensus patterns built in

✗ Limitations

- Conversation-centric model unintuitive for non-chat workflows
- v0.2 → v0.4 was a near-complete API rewrite
- Conversation history grows unboundedly without management
- Research-focused — fewer production deployment patterns

Use when: Code generation and debugging tasks; multi-agent debate/consensus; Microsoft Azure-centric deployments; research prototyping requiring high configurability.

6. Amazon Bedrock Agents

Amazon Bedrock Agents

Type: Fully managed agentic service (AWS) **License:** Proprietary **GA:** November 2023 **Pricing:** Per-token + per-session

Bedrock Agents is a **configuration-driven managed service** — no orchestration code, just API calls and Lambda functions. Developers define **Action Groups** (tools as OpenAPI 3.0 schemas backed by Lambda), **Knowledge Bases** (managed RAG backed by Amazon OpenSearch Serverless or Aurora pgvector, with automatic Titan Embeddings ingestion — cross-refs Docs 1 & 3), **Session Management** (DynamoDB-backed state; 10-min default TTL, max 24h), and **Guardrails** (content filters, PII redaction, prompt injection detection, topic deny lists).

```
import boto3
bedrock_agent = boto3.client("bedrock-agent", region_name="us-east-1")
agent = bedrock_agent.create_agent(
    agentName="KnowledgeAssistant",
    foundationModel="anthropic.claude-3-sonnet-20240229-v1:0",
    instruction="Answer from the knowledge base. Cite sources when relevant.",
    idleSessionTTLInSeconds=600
)
# Associate Knowledge Base (OpenSearch Serverless vector store)
bedrock_agent.associate_agent_knowledge_base(agentId=AGENT_ID, knowledgeBaseId=KB_ID)
# Invoke at runtime (streaming supported)
response = boto3.client("bedrock-agent-runtime").invoke_agent(
    agentId=AGENT_ID, agentAliasId=ALIAS_ID,
    sessionId=session_id, inputText=user_query
)
```

✓ Strengths

- Fully managed: no orchestration code, AWS handles scaling & HA
- Native AWS integration: S3, Lambda, DynamoDB, IAM, VPC, KMS
- Enterprise security: Guardrails, VPC endpoints, AWS Config compliance
- Managed RAG: end-to-end ingestion → embedding → retrieval
- SLA-backed; auto-scales from zero to high concurrency

✗ Limitations

- Vendor lock-in — tightly coupled to AWS; hard to migrate
- Limited orchestration flexibility vs. open-source
- Restricted to Bedrock-supported foundation models
- Cost unpredictability in agentic loops without token budgets
- Less granular debugging vs. LangSmith

Use when: AWS-native enterprises prioritizing operational simplicity; regulated industries needing managed security/compliance; teams without deep MLOps expertise; managed RAG + guardrails more valuable than framework flexibility.

7. Head-to-Head Framework Comparison Matrix

Dimension	LangChain	LangGraph	CrewAI	AutoGen	Bedrock Agents
Architecture Paradigm	Chain / Agent loop	Directed stateful graph	Role-based crew	Conversational agents	Managed API service
State Management	Limited (buffer)	Native typed state	Per-agent memory	Conversation history	Managed (DynamoDB)
Multi-Agent Support	Via LangGraph	Native sub-graphs	Core feature	Core feature	Preview (2024)
Tool Integration	700+ integrations	All LangChain tools	LangChain + custom	Custom functions	Lambda + OpenAPI
Human-in-the-Loop	Manual interrupt	Native breakpoints	Task-level flag	human_input_mode	Return-control action
Checkpointing	None native	SQLite/Postgres/Redis	None native	None native	Managed DynamoDB
Code Execution	Via tool	Via tool node	Via tool	Native Docker sandbox	Lambda only
Observability	LangSmith (native)	LangSmith + Studio	Verbose logs only	Custom logging	CloudWatch + Trace
Learning Curve	Low	Medium-High	Low-Medium	Medium	Low (config-driven)
Production Readiness	Medium	High	Medium	Medium (research)	High (SLA-backed)
Community	Very large ~95k ★	Growing ~12k ★	Large ~28k ★	Large ~38k ★	AWS forums/docs
Licensing	MIT (open-source)	MIT (open-source)	MIT (open-source)	MIT (open-source)	AWS Commercial

Dimension	LangChain	LangGraph	CrewAI	AutoGen	Bedrock Agents
Cloud Dependency	None (self-hosted)	None (self-hosted)	None (self-hosted)	None (self-hosted)	AWS required
Typical Task Latency	10–45s	10–60s	15–90s	20–120s	8–30s (managed)

Selection Heuristic for Knowledge Assistants

For RAG-centric knowledge assistants: **LangGraph** (production-grade, explicit state, checkpointing + LangSmith tracing) or **LangChain** (rapid prototyping). **Bedrock Agents** for AWS-native with managed RAG. **CrewAI** for role-specialized multi-agent tasks. **AutoGen** for code-generation-heavy workflows.

8. Orchestration Patterns

Pattern	Mechanism	Best For	Key Trade-offs
ReAct (Reasoning + Acting)	Interleaves Thought → Action[tool] → Observation[result] in a loop until final answer. Grounds reasoning in external observations.	General-purpose tool use; interpretable traces; any tool set	+ Interpretable trace; simple to implement. – Greedy; no backtracking; ~40–120 tokens overhead per step
Plan-and-Execute	Planner LLM call produces structured task list upfront; Executor agent works through steps; optional Replanner revises on failures.	Complex multi-step tasks with knowable structure; cost-predictable workflows	+ Predictable execution; parallelizable steps. – Brittle if early steps fail; replanning adds 2–5s latency
Reflection / Self-Critique	After generation, a Critic agent (or same agent) evaluates output quality; critique fed back for N revision rounds or until quality threshold met.	Code generation; writing tasks; factual accuracy improvement	+ +15–25% quality on benchmarks. – 2–3x token cost; can converge on local optima
Multi-Agent Debate	N agents independently generate solutions; structured debate rounds; Judge agent or majority vote selects winner. Reduces hallucination ~14% vs. single agent (Du et al., 2023).	Factual verification; controversial or ambiguous decisions	+ Diverse perspectives reduce single-model blind spots. – 3–5x token cost; 30–120s per task
Router / Dispatcher	Lightweight classifier routes queries to specialized agents or tools (e.g., technical → code agent; HR → policy agent; math → calculator).	Multi-domain systems; cost-tiered model routing; clear intent boundaries	+ Low latency; cost-efficient; maintainable. – Routing errors are silent; classifier adds ~200–500ms
Supervisor	Supervisor agent receives task, assigns sub-tasks to Worker agents, collects results, synthesizes final response. Workers do not communicate directly.	Complex tasks with multiple specialized workers; parallel sub-task execution	+ Clean separation of concerns; workers can use smaller/cheaper models. – Single point of failure; 2–3x more LLM calls

9. Tool Use and Function Calling

Tool use is the mechanism by which agents interact with the external world. Modern LLMs expose a **function calling API** (OpenAI) or **tool use API** (Anthropic) that returns structured JSON specifying which tool to call and with what arguments — the framework then executes the function and feeds the result back as the next message. The LLM never executes tools directly; it only outputs structured invocation requests.

```
# Tool schema (OpenAI convention) - precision in description drives LLM selection accuracy
TOOL_SCHEMA = {"name": "search_vector_database",
  "description": "Search internal knowledge base via semantic similarity. "
    "Use when query requires specific facts from internal documents.",
  "parameters": {"type": "object", "properties": {
    "query": {"type": "string", "description": "Query rephrased for semantic retrieval"},
    "top_k": {"type": "integer", "default": 5, "description": "Number of results (1-20)"},
    "filter": {"type": "object", "description": "Optional metadata filter, e.g. {doc_id: 3}"},
    "required": ["query"]}}
```

```
# Structured error handling wrapper
def execute_tool_safely(name, args):
    try:
        return ToolResult(success=True, content=str(tool_registry[name](**args)))
    except ArgumentError as e:
        return ToolResult(success=False, content=f"ArgumentError: {e}. Check required params.")
    except TimeoutError:
        return ToolResult(success=False, content="Tool timed out (30s). Try a simpler query.")
```

Best Practice	Implementation	Impact
Precise descriptions	Define <i>when</i> to use a tool, not just what it does	+20–35% tool selection accuracy
Minimal required fields	Mark only truly mandatory params as required; use defaults	~40% fewer JSON parse errors and agent retries

Best Practice	Implementation	Impact
Narrow scope (atomic tools)	One tool per capability; avoid multi-function tools	Reduces ambiguity; LLMs struggle to select sub-functions
Idempotent read tools	Safe to retry; write tools must be idempotent	Prevents duplicate side effects on agent retries
Encapsulate RAG complexity	Wrap full retrieval pipeline (query transform + hybrid search + rerank) as single tool (Docs 1 & 3)	Agent only passes natural language query; no retrieval mechanics exposed

10. Production Considerations

10.1 Monitoring & Observability

Tool	Framework Support	Key Capabilities	Cost
LangSmith	LangChain, LangGraph (native)	Full trace trees, token counts per step, latency breakdown, regression datasets, prompt versioning, human feedback	Free 5k traces/mo; \$39/mo Developer
Arize Phoenix	Framework-agnostic (OpenTelemetry)	Trace visualization, span-level evals, embedding drift detection	Open-source self-hosted or cloud free tier
AWS CloudWatch + X-Ray	Bedrock Agents (native)	Agent trace view, Lambda logs, cost attribution per session	Standard CloudWatch pricing; ~\$0.05/trace X-Ray

10.2 Cost Management

Agentic loops can consume 10–50x more tokens than simple QA. Key controls: (1) **Max iterations** — always set `max_iterations / max_round` (production default: 10–15); uncapped agents loop indefinitely on ambiguous tasks. (2) **Model tiering** — GPT-4o-mini/ Claude Haiku for routing and tool selection; GPT-4o/ Claude Sonnet for final synthesis; 60–75% cost reduction with minimal quality impact. (3) **Prompt caching** — Anthropic Extended Cache and OpenAI automatic caching reduce repeated-context cost by 50–90%. (4) **Token budget tracking** — alert at 80% context window; apply sliding window or summarization beyond threshold.

10.3 Security Considerations

- **Sandboxed code execution** — Always isolate in Docker (read-only filesystem, no network, resource limits: 512MB RAM, 2 CPU, 60s). Never execute agent-generated code on the host process.
- **Prompt injection via tool outputs** — Tool results (web search, user files, DB content) may contain adversarial instructions. Mitigations: prefix with "Tool Result [untrusted]:"; apply a classifier to scan before injecting into context, limit output length to prevent context flooding.
- **Credential management** — Never pass API keys in LLM-visible parameters. Use environment variables, AWS Secrets Manager, or Vault. Agents access secrets through the execution environment only.
- **PII accumulation** — Conversation state may accumulate PII across turns. Apply scrubbing before logging; use Bedrock Guardrails PII redaction or LangSmith field masking for managed deployments.

10.4 Testing & Latency Optimization

Test Layer	Scope	Tools / Methods	Target
Unit Tests	Individual tools, prompt templates, output parsers	pytest + mock LLM responses (fixed JSON)	100% branch coverage on all tools
Integration Tests	Full agent run on known inputs; verify tool call sequence	LangSmith eval datasets; VCR cassettes for tool calls	20–30 canonical test cases
LLM-as-Judge Eval	Answer quality, faithfulness, tool selection correctness	RAGAS (Doc 1); LangSmith evaluators; G-Eval	Faithfulness ≥ 0.85 ; answer relevance ≥ 0.80
Adversarial Tests	Prompt injection; max-iteration triggers; unanswerable Qs	Red-team test suite; fuzzing tool inputs	All known attack vectors covered

Latency optimization: Enable token-level streaming (LCEL `astream`; Bedrock streaming) to reduce time-to-first-token from ~10s to <1s. Execute independent tool calls in parallel (LangGraph parallel nodes; target <500ms per tool in critical path). Pre-load tool schemas and vector DB connections at startup; avoid per-request initialization. For high-volume deployments, use Bedrock Agents provisioned throughput or LangGraph Platform to eliminate cold-start latency.

Document 6 of 6 — Agentic AI Frameworks | Senior AI Engineer Technical Assessment Sample Corpus. Cross-references: Doc 1 (RAG Architecture Patterns), Doc 3 (Vector Database Comparison). Questions about frameworks not covered here (Haystack Agents, Semantic Kernel, Flowise) or specific benchmark datasets not present in this document should be handled as unanswerable by the Knowledge Assistant.